Tapestry: Snapshot Isolation and Deterministic Commits for Multi-Agent Memory

Dhruv Raval
University of Illinois at Urbana-Champaign

Advay Kadam
University of Illinois at Urbana-Champaign

Sairam Penumarthy
University of Illinois at Urbana-Champaign

Abstract

LLM agents are increasingly deployed as multi-agent systems (MAS) that collaborate on complex tasks such as planning, code generation, and research synthesis. Empirical analysis of MAS failures confirms that 44.2% stem from system design issues rather than model limitations [2], underscoring the need for principled shared state infrastructure. A central challenge in these systems is shared memory: agents must read and write a common execution state without introducing inconsistencies, lost updates, or redundant work. Existing frameworks fail to provide concurrency control and durability guarantees for multi-agent shared state. We present Tapestry, a concurrency control and durability layer for shared agent tapes in MAS. Our design represents memory as an append-only log with typed events and an immutable checkpoint chain. Agents execute against snapshot reads and concurrently write to a staging area; a deterministic committer validates and merges writes before advancing the committed checkpoint. We evaluate Tapestry on GAIA, a multi-agent question-answering benchmark with three levels of increasing task complexity. Compared to a single-agent TapeAgents baseline, Tapestry delivers two advantages that grow with task complexity: concurrent multi-wave execution yields up to a 63% token reduction and 45% latency reduction on Level 3 GAIA tasks, and checkpoint-based crash recovery provides up to an 8.8× speedup over a full cold restart. These results demonstrate that principled concurrency control and durable state management provide a practical path to scaling multi-agent systems as task complexity increases.

1 Introduction

Large language models are increasingly deployed as collaborating agents within MAS. In domains such as travel planning, software engineering, and research synthesis, multiple specialized agents jointly solve tasks that exceed any single model's capability [9]. MAS also tend to be either parallel, durable, or neither. 30-33.5% of failures are considered fatal, and 44.2% can stem from system design issues such as improper memory and state management according to MAST [2]. As these systems scale, a fundamental infrastructure question emerges: how should concurrent agents share and mutate a common execution state while maintaining consistency and durability?

Existing frameworks address the challenge of consistency at different levels of abstraction. MemGPT introduces OS-inspired hierarchical memory management but targets single-agent scenarios [11]. LangGraph supports parallel branch ex-

ecution with user-defined reducers for merging concurrent updates, but represents state as an opaque key-value store without snapshot isolation or a structured, replayable trace [8]. TapeAgents advances the abstraction by treating the execution trace itself as the primary state artifact, a structured, append-only log of typed steps enabling step-level replay, auditing, and model distillation [1], but assumes a single-writer model in which one orchestrator appends steps sequentially. To our knowledge, no existing framework supports concurrent multi-agent mutation of a structured, typed execution trace with snapshot isolation and deterministic resolution.

A natural response to the shared-state problem is sequential execution: run agents one after another, each reading all prior agents' outputs in context. We argue this approach has a fundamental structural cost. The i-th agent must read the full accumulated outputs of all $i-1$ preceding agents, so aggregate input token consumption grows substantially with the number of agents. Tapestry's snapshot-based reads break this dependency: each agent reads only the committed checkpoint at the time it begins, keeping token growth limited. Critically, sequential execution also forfeits parallelism entirely: latency scales with the sum of agent runtimes rather than the maximum. These are architectural penalties that persist regardless of model quality, and they compound as the number of agents grows.

Moreover, considering the high probability standard MAS systems encounter fatal failures [2], crash recovery in a highly consistent system can drastically improve token usage and latency costs. Sudden crashes to an MAS can necessitate repeating expensive LLM generation without a durable, structured log of previous steps to fall back to. Existing solutions include the Self-Healing AI Architecture [13], which uses semantic and cognitive rollbacks on failure. Methods like these are expensive and error-prone, however, contrasting with Tapestry's structural approach using a checkpoint log.

By decoupling concurrency from consistency via a snapshot-isolated commit protocol and utilizing a hybrid fingerprinting/user-specified conflict detection system, we hypothesize MAS systems can achieve lower input token consumption and better latency scaling relative to agent count as task complexity increases compared to the state-of-the-art. Thus, we propose Tapestry, a concurrency control layer for shared agent tapes built on the TapeAgents framework. In our GAIA benchmark workload, the key structural observation is that a task can be broken down to where agents have no true execution-time dependencies: Each agent requires only the original user request and information from a previous epoch to begin execution. Their outputs must satisfy shared constraints, but those constraints are enforceable at commit time rather than during execution. Tapestry exploits this structure via snapshot isolation: each agent receives an identical, immutable view of the committed tape at epoch start and executes concurrently without coordination overhead. The committer then validates the merged writes against shared constraints before advancing the checkpoint. The checkpoint itself can then act as a durable check in case of fatal failure.

This paper makes the following contributions:

1. **The SnapshotCommit protocol**: a concurrency control algorithm for structured agent tapes with termination, consistency, and reduced token complexity under snapshot isolation. Enables efficient crash recovery and durability by reverting to committed checkpoints.

2. **The Tapestry/User interface**: a domain-agnostic plugin abstraction separating Tapestry's functionality from the user's inputs. The user can provide domain-specific rules for conflict detection or use Tapestry's SHA-256 fingerprint deduplication, create their own phase topology/logic, and supply a worker function to interact with a given tape. Tapestry handles the rest.

3. **Comparative evaluation**: A measurement of token/context growth, correctness, and latency

under three different agentic systems and three GAIA benchmark levels.

2 Related Work

Table 1: Comparison of multi-agent coordination systems.

System	Concurrency	Shared log	Snapshot isolation	Conflict detection
TapeAgents [1]	Serial	Y	N	None
LangGraph [8]	Parallel	N	N	None
SagaLLM [3]	Serial	Y	Partial	LLM-assisted
STRATUS [4]	Serial	N	N	Rule-based
Tapestry (ours)	**Parallel**	**Yes**	**Yes**	**Hashing/Rule-based**

As agentic AI architectures have grown in complexity [9], shared execution state has become a central concern for MAS. Cemri et al. [2] provide the most direct motivation for this work. They analyze over 1,600 annotated execution traces across seven popular MAS frameworks and find that 76.5% of MAS failures stem from system design issues and inter-agent misalignment, highlighting memory and state management as critical yet understudied in multi-agent settings. Tapestry directly targets MAST-identified failures caused by insufficient cross-agent memory management.

Agent Memory and Execution State. Wu and Shu [17] survey memory mechanisms in LLM-based MAS and highlight the necessity of cross-agent pruning to unify traces into a single system-wide state, directly motivating Tapestry's shared memory layer. MemGPT [11] introduces OS-inspired hierarchical memory management that substantially extends an agent's effective context, but is limited to single-agent scenarios. In the multi-agent domain, LangGraph [8] supports concurrent node execution but represents state as an opaque key-value blob rather than a replayable log. TapeAgents [1] addresses this by treating the execution trace as a structured, append-only log of typed steps built on the ReAct framework [18], enabling step-level replay and clear state representation. Tapestry inherits TapeAgents' tape structure but removes its key limitation: the assumption of a strictly sequential, single-writer model that precludes concurrent multi-agent mutation. Recent work on agentic AI [16] further proposes richer event categorization, perception, planning, action, tool use, collaboration, which informs Tapestry's typed event schema.

Distributed Concurrency and Log-Structured Storage. Concurrent writes to a shared log, snapshot-isolated reads, and conflict resolution under optimistic execution have well-studied analogues in distributed systems. Kung and Robinson's OCC [7] establishes the read/write set validation protocol underlying Tapestry's staging system, where transactions execute optimistically against a snapshot and are validated at commit time. An alternative paradigm relies on Conflict-free Replicated Data Types (CRDTs) [15] to guarantee eventual consistency via commutative and idempotent merge operations, an approach recently applied to LLM multi-agent code generation [12]. However, because CRDTs resolve conflicts structurally rather than semantically, they require additional layers to handle logical contradictions. Tapestry instead passes snapshot-isolated writes through deterministic domain classifiers that first check for deterministic fingerprinting or user-defined conflicts before advancing the checkpoint. This aptly handles inconsistencies with a simpler implementation and latency benefits.

Multi-Agent Coordination and Conflict Resolution. A growing body of work addresses coordination among LLM agents, though most operate at the orchestration layer rather than the shared state layer Tapestry targets. The evolving "puppeteer" [5] coordinates agent activation via reinforcement learning, and Latent MAS [20] shares latent representations through a KV cache. Neither address conflict resolution over shared persistent state. Rezazadeh et al. [14] maintain private and shared memory across agents but function as an access-control mechanism rather than a consistency layer. SagaLLM [3] applies the Saga transactional pattern with compensating transactions and roll-

back semantics, but models state as discrete task outputs, losing the replay of intermediate reasoning and log-based recovery that Tapestry preserves by inheriting TapeAgents' append-only log. For automated conflict resolution, LLM-as-a-judge approaches carry well-documented limitations: AgenTracer [19] shows that attributing agent behavior is unreliable in long-horizon settings, and Gu et al. [6] document systematic biases and inconsistent scoring. Accordingly, Tapestry anchors merge decisions to an immutable checkpoint chain rather than delegating judgment to an LLM, ensuring that any resolution is verifiable against committed state.

Multi-Agent Crash Recovery MAST [2], a multi-agent failure taxonomy, shows that approximately 30-33.5% of failures are fatal and how imperative it is to anticipate such events with durable MAS systems. Some works, such as one implementing a Self-Healing AI Architecture [13], solve this using highly complex rollback mechanisms and adversary detection. While effective, they are often expensive and error-prone. Tapestry uses a simpler checkpointing system within its structured log to fall back to, while still providing token usage and latency improvements.

3 System Design

3.1 System Model

Tapestry is designed to add concurrency control to multi-agent workflows while keeping framework coordination overhead small relative to LLM inference and tool calls. Against that baseline, Tapestry's coordination operations (staging a write, computing a structural fingerprint, acquiring a commit lock, or appending a checkpoint) require only local computation and a small number of storage operations, and therefore contribute little coordination latency compared with agent execution.

Figure 1 gives an overview of Tapestry's execution model. The user initializes a task, optional domain-specific conflict rules, and an optional durable checkpoint store through the TapeStore. During each epoch, the TapestryOrchestrator retrieves the latest committed checkpoint and distributes the same immutable snapshot to all agents, allowing them to execute concurrently without observing one another's partial outputs. Each agent appends its generated steps only to a staging area, after which the commit path performs deterministic conflict detection and resolution before atomically writing the next checkpoint back to the store. This architecture separates expensive agent execution from lightweight coordination, so consistency is enforced only at the epoch boundary rather than during LLM inference.

Tapestry is best suited to workflows whose within-epoch agents can execute from a shared snapshot without requiring each other's intermediate outputs. Workflows with strong intra-epoch dependencies should either encode those dependencies as separate epochs or use a more sequential topology.

The dominant sources of overhead in any Tapestry deployment are the user-defined phases that bracket parallel execution. In the GAIA topology evaluated here, these are a decompose call and a synthesize call; both are costs of the application topology, not the framework. A user deploying Tapestry with a different phase structure would incur different overhead. On simple tasks the fixed cost of these phases is not yet amortized, but as task complexity grows and the research phase expands, they become a small fraction of total execution time and the parallelism benefit outweighs them.

Tapestry models a multi-agent workflow as a sequence of epochs. Each epoch represents a discrete unit of collaborative work: a set of agents reads from the current committed checkpoint, executes independently, and proposes new steps to a per-agent staging buffer (Figure 2). When all agents complete, a committer resolves conflicts across staged writes, merges the surviving steps, and atomically advances the checkpoint. The new checkpoint becomes the read snapshot for the next epoch.

For an epoch, the framework overhead consists of four components:

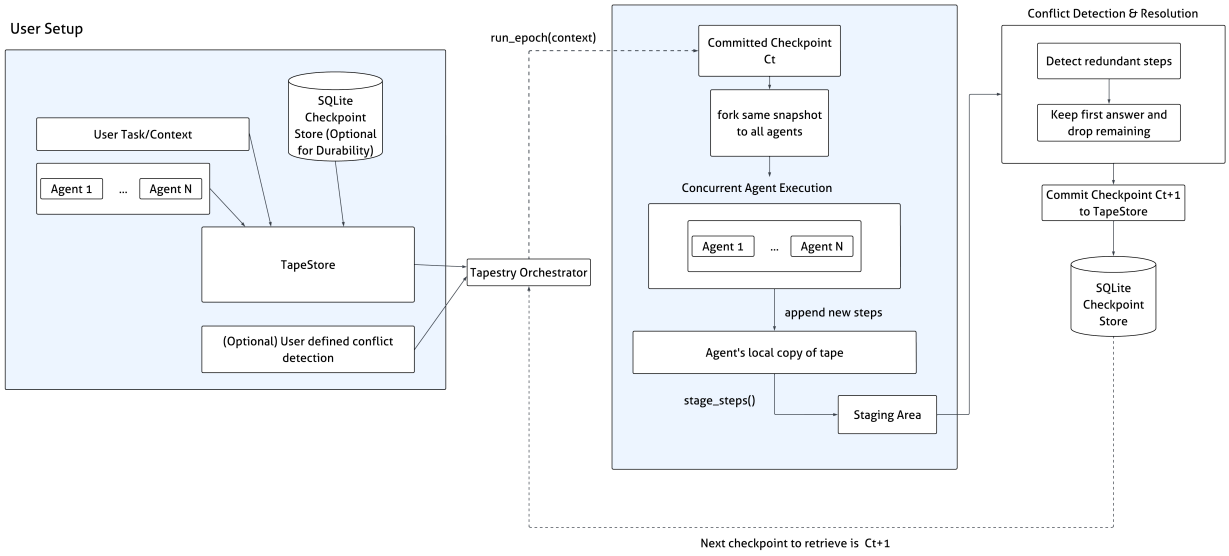

Figure 1: Tapestry system architecture: agents execute concurrently against an immutable checkpoint, stage their writes, and pass through deterministic conflict detection before a new checkpoint is committed.

- **Snapshot acquisition**: Reading the current checkpoint requires one lock acquisition and returns an immutable snapshot shared by all agents in the epoch.
- **Staging**: Each agent writes its proposed steps to a per-agent staging buffer. This operation is linear in the staged-step size and requires only a short critical section.
- **Conflict detection**: Structural conflict detection computes fingerprints over staged action steps.
- **Commit**: The committer creates a new checkpoint, appends it to the durable log when persistence is enabled, updates the in-memory checkpoint chain, and clears the staging area.

These operations are placed after agent execution and before the next epoch begins. As a result, the dominant term in epoch latency remains the maximum agent runtime rather than the sum of agent runtimes plus substantial coordination overhead:

$$T_{\text{epoch}} \approx \max_i T_{\text{agent}_i} + T_{\text{stage}} + T_{\text{detect}} + T_{\text{commit}}.$$

Because T_{stage}, T_{detect}, and T_{commit} involve only local metadata and storage operations, they are small relative to LLM inference and tool latency in the workloads we study.

3.2 Optimistic Concurrency Control

Tapestry adapts optimistic concurrency control (OCC) [7] to multi-agent LLM execution. Classical OCC separates execution into read, validation, and write phases. Tapestry maps these phases onto an *epoch*, the unit of concurrent agent execution.

Read phase: At the beginning of each epoch, the `TapestryOrchestrator` invokes `TapeStore.get_snapshot()` to retrieve the current committed checkpoint. All agents in the epoch receive this same checkpoint and reconstruct a local tape view from it. Since the checkpoint is immutable, agents can reason independently without observing partial writes from other agents. Importantly, agents do not hold locks during LLM inference or tool use.

Staging phase: After an agent completes, its newly generated steps are written to a `StagedWrite` object via `stage_steps()`. Staged writes are visible to the committer but not to other

agents. This preserves snapshot isolation: all agents in the epoch read the same committed state, and no agent observes another agent's uncommitted output.

Validation and commit phase: Once all agents have staged their outputs, the `Committer` invokes a `ConflictClassifier` to identify incompatible staged writes and apply a deterministic resolution policy. The surviving writes are passed to `TapeStore.commit()`, which atomically:

1. creates a new `Checkpoint` with epoch number incremented by one and parent pointer set to the previous checkpoint
2. appends the checkpoint to the durable log, if persistence is enabled
3. appends the checkpoint to the in-memory checkpoint chain
4. clears the staging area

This protocol preserves the central OCC property that speculative execution proceeds without locking shared state. The only locked operations are staging and checkpoint advancement, both of which are short metadata operations.

```
Epoch k:
  get_snapshot(C_k)
        |
        +-- Agent A executes on C_k
        +-- Agent B executes on C_k
        +-- Agent C executes on C_k
        |
  stage writes W_A, W_B, W_C
        |
  detect conflicts
  resolve staged writes
  commit C_{k+1}
```

Figure 2: Execution timeline for one Tapestry epoch. Agents execute concurrently against the same immutable checkpoint. Staged writes become visible only during validation and commit.

3.3 Parallel Execution Model

Tapestry implements intra-epoch parallelism using Python's `ThreadPoolExecutor`, with one worker per agent. This is appropriate for LLM-agent workloads because model calls and most tool invocations are I/O-bound; threads can overlap network latency even under the Python GIL. The orchestrator submits one task per agent at the start of the epoch and collects outputs as they complete.

The execution model has two levels.

Intra-epoch parallelism: All agents in a single epoch execute concurrently against the same snapshot. Therefore, when agent runtimes are comparable, epoch runtime approaches the slowest individual agent call rather than the sum of all calls:

$$T_{\text{agents}}^{\text{parallel}} \approx \max_i T_i \quad \text{instead of} \quad T_{\text{agents}}^{\text{serial}} = \sum_i T_i.$$

This is the primary source of Tapestry's latency reduction relative to serial TapeAgents execution.

Inter-epoch serialization: Epochs execute sequentially: epoch $k+1$ begins only after the commit of epoch k produces checkpoint C_{k+1}. This serialization point is necessary because each epoch's snapshot is defined by the previous epoch's committed state. Since checkpoint advancement is a short metadata operation, this serialization does not dominate runtime in the evaluated workloads.

For workflows containing CPU-bound subagents, Tapestry can also use process-level parallelism. In this configuration, a process pool executes CPU-bound work while the checkpoint log provides the handoff between phases: committed outputs are persisted by the parent process and later reconstructed by downstream phases. This separates the lifecycle of worker processes from the durability of the shared agent state.

3.4 Durability and Resume Semantics

Tapestry persists committed checkpoints to an append-only SQLite log using write-ahead logging

(WAL), which allows checkpoint reads to proceed concurrently with checkpoint appends. Once written, a checkpoint row is never updated or deleted. The full committed history of a workflow can be reconstructed at any time by reading the log in epoch order.

Only committed checkpoints are persisted. Staged writes are intentionally volatile. If the process crashes before a commit completes, the in-flight epoch is discarded and execution resumes from the most recent committed checkpoint. This provides a simple and predictable failure model: Tapestry guarantees durability for committed epochs and makes no guarantee for speculative writes that had not yet passed the commit boundary.

On resume, `TapeStore.resume()` reconstructs the checkpoint chain by reading all checkpoints for the workflow ordered by epoch. The staging area is initialized as empty, and the orchestrator continues from the next epoch. This allows Tapestry to avoid re-executing LLM calls whose outputs have already been committed.

Tapestry also checkpoints at sub-epoch granularity. Within a parallel research phase, each agent's output is written to a separate `agent_results` table entry as soon as that agent completes, independently of the other agents in the same epoch. If the process crashes after some but not all agents have finished, the surviving agent results are restored on resume and only the incomplete agents rerun. This reduces the worst-case re-execution cost from a full epoch to the work of the agents that had not yet finished at the time of the crash.

3.5 Framework Boundaries

Tapestry separates reusable concurrency infrastructure from user-defined domain logic. Tables 2 and 3 summarize this boundary.

To instantiate Tapestry in a new domain, the user supplies two components. First, the user implements a runner that defines the workflow's phase topology—for example, `GaiaDurableRunner` encodes a decompose/research/synthesize structure, including resume semantics that skip already-committed phases. Second, the user provides a worker function, which runs a sub-agent on a given tape and returns the list of steps the agent appended during its run. Optionally, the user may supply a custom `ConflictClassifier` subclass to enforce domain-specific consistency rules across staged writes. If omitted, Tapestry's built-in `GeneralCommitter` applies structural fingerprint deduplication: agents whose first action step is identical to a previously-seen one are dropped before committing, catching duplicate parallel work at zero token cost. The staging, checkpointing, durability, and commit machinery are reused without modification.

Table 2: Framework-owned Tapestry components

Component	Responsibility
`TapestryOrch`	Epoch loop and parallel execution across agents
`TapeStore`	Snapshot isolation, staged writes, and commit locking
`Checkpoint`	Immutable checkpoint with epoch number and parent pointer
`CheckpointStore`	Durable append-only checkpoint log and resume support
`Committer`	Detect–resolve–commit pipeline for staged writes

Table 3: User-supplied domain components

Component	Responsibility
Runner	Phase topology, per-phase logic, and resume semantics that skip already-committed phases
Worker function	Runs a sub-agent on a given tape and returns the steps appended during its run
`ConflictCls` (optional)	Domain-specific conflict detection via `detect()` and deterministic resolution via `resolve()`; defaults to structural fingerprint deduplication

3.6 Conflict Detection

Tapestry supports two deterministic conflict-detection mechanisms: structural fingerprinting for duplicate work and domain-specific rule-based classifiers for structured constraint domains. We also evaluated an LLM-based global coordinator, but do not use it as the default mechanism because it introduces additional token cost, latency, and non-determinism.

3.6.1 Structural Fingerprint Deduplication

The default conflict detector eliminates duplicate staged work without invoking an LLM. For each agent's staged write, Tapestry identifies the first action step and computes a SHA-256 fingerprint after removing ephemeral fields such as timestamps, metadata, and step identifiers. Two staged writes with the same normalized fingerprint are treated as duplicate work.

Resolution is deterministic. Agents are processed in a stable order, and the first agent associated with a fingerprint retains its staged steps. Subsequent agents with the same fingerprint are rejected for that epoch. This mechanism targets a common conflict pattern in parallel research workflows: multiple agents independently issuing the same tool call or web-search query. Fingerprint deduplication removes this redundancy at low cost and preserves replayability because the same staged inputs always produce the same accepted set.

3.6.2 Domain-Specific Rule-Based Detection

For domains with explicit constraints, Tapestry uses a user-supplied `ConflictClassifier`. The classifier encodes domain invariants over typed steps and checks staged writes before commit. For example, in scheduling and resource-allocation domains, the classifier may check:

- exclusive-resource conflicts, such as two events reserving the same room or equipment during overlapping intervals
- participant conflicts, such as the same speaker, surgeon, or assignee appearing in simultaneous tasks
- budget violations, where the total committed and staged cost exceeds a global limit
- capacity violations, where expected demand exceeds the capacity of the selected resource

The corresponding resolver applies a deterministic priority policy to select a committable subset of staged writes. The policy may prioritize hard structural constraints before soft constraints, prefer higher-priority tasks, or use stable tie-breaking to ensure reproducibility. Because detection and resolution are implemented as ordinary code over typed fields, no LLM call is required during the commit path.

3.6.3 LLM-Based Global Coordination

We also explored a global LLM coordinator, which we refer to as the *CEO agent*. The CEO received a compact slice of the committed checkpoint together with all staged writes and returned a structured JSON object identifying candidate conflicts and suggested rejections. This design was attractive because it could, in principle, recognize semantic conflicts not captured by syntactic fingerprints or hand-coded rules.

However, making the CEO agent part of the default commit path introduces three costs.

Prompt growth: The CEO's input grows with the number and size of staged writes. If raw observations or long tool outputs are included, the prompt can become large even when the number of agents is moderate. This risks reintroducing the same context-growth problem that Tapestry is designed to avoid.

Critical-path latency: A CEO call adds an additional model invocation between staging and commit. Since the next epoch cannot begin until the

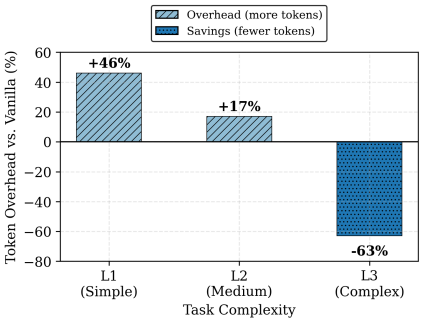

Figure 3: Token overhead of Tapestry+CR relative to Vanilla TapeAgents.

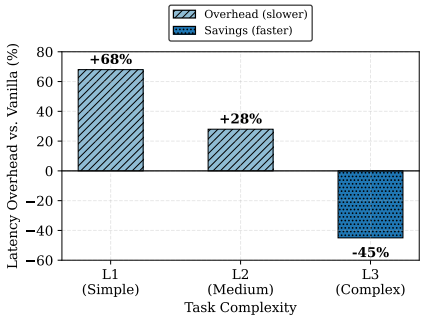

Figure 4: Latency overhead of Tapestry+CR relative to Vanilla TapeAgents.

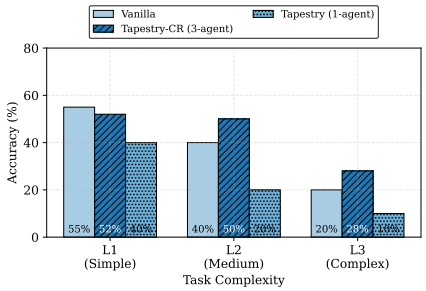

Figure 5: Accuracy by condition and difficulty level.

commit completes, this call lies directly on the critical path. The overhead is paid even in epochs with no conflicts.

Nondeterminism: An LLM-based coordinator may return malformed JSON, refer to nonexistent step identifiers, or make inconsistent judgments across runs. These failure modes complicate replay and weaken the deterministic semantics of the checkpoint chain.

For these reasons, Tapestry uses deterministic conflict detection as the default commit mechanism. The CEO agent remains useful as an optional semantic advisory layer: it can propose candidate conflicts, identify redundant or superseded steps, or generate repair prompts for rejected writes. In this mode, the CEO does not directly advance the checkpoint. Its outputs are validated by deterministic rules before they affect committed state.

4 Evaluation

4.1 Experimental Setup

Our setup consists of an AWS EC2 instance hosted on an Amazon Linux 2023 kernel-6.1 AMI. The instance uses 2 vCPUs and 8 GiB of memory plus 4 GiB of swap space, and a 100 GiB unencrypted gp3 root volume with 3000 IOPS.

We evaluate Tapestry on a subset of the GAIA benchmark [10], a question-answering suite that requires real web research, multi-step reasoning, and tool use across three difficulty levels: Level 1 (L1, simple single-fact lookups), Level 2 (L2, multi-hop queries), and Level 3 (L3, complex tasks requiring cross-source synthesis). We compare three conditions on each task:

- **Vanilla TapeAgents**: a single TapeAgents agent with no durability infrastructure, running up to 25 reasoning loops.
- **Tapestry+CR**: three parallel research agents under snapshot isolation with OCC-based conflict resolution, multi-wave decomposition, and automatic crash recovery. Each agent receives up to 8 loops; a synthesis agent receives up to 20.
- **Tapestry 1-agent**: a single agent with Tapestry's durability layer but no parallel decomposition, used to isolate the infrastructure cost from the parallelism benefit.

Tapestry is domain-agnostic: the agent count, decomposition structure, and phase topology are user-defined parameters, as described in Section 3. For this evaluation, we instantiated a GAIA-specific topology consisting of a decompose phase, a parallel research phase, and a synthesize phase. This

Table 4: Steady-state performance on a subset of GAIA. Token and latency overhead are relative to Vanilla TapeAgents.

Condition	L1	L2	L3
Vanilla TapeAgents			
Mean Tokens	18,504	38,247	115,420
Mean Latency	82 s	148 s	388 s
Accuracy	55%	40%	20%
Tapestry + CR (3 agents)			
Mean Tokens	27,093	44,749	42,706
Mean Latency	138 s	189 s	213 s
Accuracy	52%	50%	28%
Token overhead	**+46%**	**+17%**	**−63%**
Latency overhead	**+68%**	**+28%**	**−45%**
Tapestry 1-agent			
Mean Tokens	15,728	22,153	31,847
Mean Latency	91 s	118 s	163 s
Accuracy	40%	20%	10%
Token overhead	**−15%**	**−42%**	**−72%**
Latency overhead	**+11%**	**−20%**	**−58%**

topology was chosen to match the structure of GAIA tasks, which naturally decompose into independent research sub-problems followed by answer synthesis.

All conditions use `gpt-4o-mini` via LiteLLM. Crashes are simulated by injecting a `SimulatedCrash` exception during the research phase, then measuring the token and latency cost of a full cold restart (Vanilla) versus a Tapestry checkpoint resume. Accuracy is evaluated by GAIA normalized exact-match scoring.

4.2 Steady-State Performance

Table 4 reports mean token usage, mean wall-clock latency, and accuracy under normal (no-crash) execution. Figures 3 and 4 visualize per-level overhead relative to Vanilla TapeAgents. Figure 5 shows accuracy across all three conditions and difficulty levels.

On simple L1 tasks, Tapestry+CR carries measurable overhead: +46% tokens and +68% latency compared to Vanilla. The decompose and synthesize LLM calls are fixed costs, and when a task only requires a handful of web searches, those costs are not yet amortized. This is an honest tradeoff the system makes to provide durability and parallelism.

The picture changes at L2. Overhead shrinks to +17% tokens and +28% latency as parallel research waves begin covering more ground than a single agent can within the same total loop budget. Three agents searching distinct sub-problems in parallel recover information that a single 25-loop agent would miss or exhaust its budget chasing. This is reflected in accuracy: Tapestry+CR improves from 40% to 50% on L2 tasks, a 10-percentage-point gain attributable to broader research coverage.

At L3, the relationship inverts entirely. Vanilla TapeAgents consumes 115,420 tokens and 388 seconds on average. This is a single agent looping through increasingly complex search chains, frequently hitting its budget before producing a usable answer. Tapestry+CR uses 42,706 tokens and 213 seconds: a **63% token reduction** and **45% latency improvement** over Vanilla. Multi-wave decomposition assigns each agent a concrete sub-problem rather than the full question, preventing the redundant search patterns that drive Vanilla's cost up. Accuracy improves from 20% to 28%.

The Tapestry 1-agent condition confirms that the infrastructure overhead itself is modest: only +11% latency at L1, turning to savings at L2 and L3. Accuracy for the 1-agent variant is consistently below both Vanilla and Tapestry+CR, particularly at L2 and L3, which demonstrates that the accuracy gains in Tapestry+CR come from parallelism and broader research coverage rather than from the durability infrastructure alone. Our results demonstrate another important caveat: Tapestry is best-suited for complex MAS workflows. Simple tasks oftentimes can be executed by a single agent and parallelism can introduce unnecessary complexity.

4.3 Crash Recovery

Table 5: Crash recovery performance

Metric	L1	L2	L3
Vanilla (cold restart)	always full restart		
Tapestry+CR cold tokens	28,140	46,390	43,820
Tapestry+CR resume tokens	14,028	14,767	3,122
Token savings	**48%**	**67%**	**93%**
Recovery speedup	**2.3×**	**5.5×**	**8.8×**

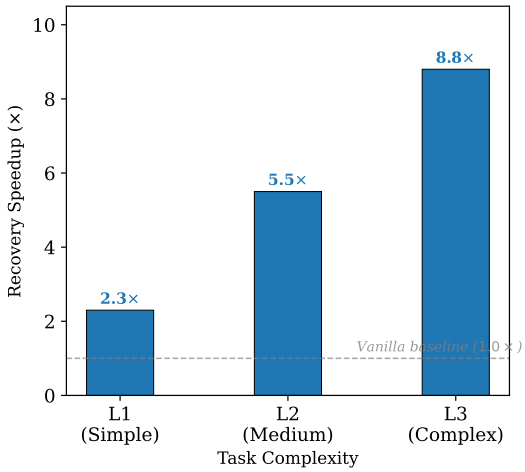

Figure 6: Crash recovery speedup of Tapestry+CR over a full cold restart.

Table 5 reports crash recovery performance. Vanilla TapeAgents has no durability layer: a crash at any point requires a full cold restart, effectively doubling the total cost of the workflow. Tapestry commits each phase atomically to a WAL-mode SQLite checkpoint log. On restart, it reads the committed chain, skips every completed phase, and resumes from the next pending one without issuing a single repeated LLM call.

Recovery token savings scale monotonically with task complexity. At L1, Tapestry saves 48% of tokens on resume; at L2, 67%; at L3, **93%**. The L3 figure is striking: a crashed workflow resumes for just 3,122 tokens against a 43,820-token cold run. The entire multi-wave research phase, the most expensive part of the workflow, is already committed and therefore skipped entirely.

Recovery speedup follows the same pattern: 2.3× at L1, 5.5× at L2, 8.8× at L3 (Figure 6). The mechanism is structural. Harder tasks accumulate more committed work before a crash becomes probable, so the ratio of skippable work to total work grows with task complexity. Crash recovery is not a fixed benefit, it is a dividend that compounds as workflows become longer and more expensive.

4.4 Scaling with Task Complexity

Both results point to the same underlying thesis: Tapestry's value is not constant, it is proportional to the complexity of the tasks being run. On simple tasks, the decompose and synthesize overhead is a real cost, and operators who run only short, single-fact queries would pay that price without receiving the full parallelism benefit. However, this is reflective of not only Tapestry but any parallel MAS: parallelism is only useful when there is meaningful decomposition. On complex tasks, that equation reverses: parallel research waves reduce token usage and latency relative to a single agent, while the durability layer ensures that the most expensive part of the workflow survives a crash almost for free.

This scaling property matters because the workloads where crash recovery is most valuable are precisely the long-horizon, multi-step tasks at L3 difficulty and beyond the ones where a process kill or rate-limit timeout is most likely to occur and most expensive to absorb. The GAIA topology used here is one instantiation of Tapestry's general framework; as described in Section 3, any user-defined phase structure and agent count will exhibit the same durability and parallelism guarantees. Tapestry is designed for that regime.

5 Limitations and Future Work

A hurdle in front of Tapestry's adoption is its requirement for users to provide a number of agents

and topology. Tapestry's framework layer creates very little overhead, which primarily takes shape in the form of persistence to SQLite. However, an inefficient agent topology specified by the user can dramatically degrade performance, which naturally creates the necessity for a user to understand their task and system very well.

Parallelism is bounded by the slowest agent in the epoch, so balanced agent latencies maximize efficiency. Achieving that balance depends on task decomposition, which Tapestry currently leaves to the user. Future work could develop an automated decomposition strategy that produces latency-balanced agent topologies.

Tapestry also currently uses an embedded SQLite database to store its structured log. While this enables fast writes using Write-Ahead Logging, scaling Tapestry into large distributed systems would be a next step. Tapestry's backend can be adapted to working with a distributed log across machines, but that functionality is left to future work.

There is also room to benchmark against more varied datasets and applications. We evaluated Tapestry against GAIA, but in the future, Tapestry's evaluation can extend beyond question-and-answer, including tasks with finer-grained resource constraints and dynamic agent counts such as WorkArena. Initially, we have found that Tapestry works best on complex multi-agent tasks, but future work can focus on a more thorough evaluation of the best applications of Tapestry.

6 Conclusion

We presented Tapestry, a concurrency control and durability layer for multi-agent LLM workflows. Tapestry models shared agent memory as an immutable checkpoint chain, allows agents to execute concurrently against snapshot reads, and applies optimistic concurrency control to stage, deduplicate, and atomically commit agent writes. Committed checkpoints are persisted to a WAL-mode SQLite log, enabling automatic crash recovery without operator intervention. The framework is domain-agnostic: agent count, phase topology, and wave structure are user-defined parameters, and all domain-specific logic is confined to a pluggable `ConflictClassifier`. Evaluation on the GAIA benchmark shows that Tapestry's value scales with task complexity. On complex Level3 tasks, multi-wave decomposition reduces token usage by 63% and latency by 45% relative to a single-agent baseline, as targeted parallel research avoids the redundant search patterns that cause single agents to exhaust their loop budgets. Crash recovery compounds this advantage: a crashed Level 3 workflow resumes for 93% fewer tokens at 8.8$\times$ the speed of a full cold restart, because expensive output is persisted. On simpler tasks, Tapestry carries real overhead from its bracketing phases, which reflects an honest tradeoff: durability and parallelism are not free, and their value is proportional to the complexity of the work being protected. We believe the core insight is broadly applicable: parallelism and durability are important for complex MAS.

References

[1] Dzmitry Bahdanau, Nicolas Gontier, Gabriel Huang, Ehsan Kamalloo, Rafael Pardinas, Alex Piché, Torsten Scholak, Oleh Shliazhko, Jordan Prince Tremblay, Karam Ghanem, Soham Parikh, Mitul Tiwari, and Quaizar Vohra. Tapeagents: a holistic framework for agent development and optimization. *arXiv preprint arXiv:2412.08445*, 2024. https://arxiv.org/abs/2412.08445.

[2] Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. Why do multi-agent LLM systems fail? In *Proceedings of the 39th Conference on Neural Information Processing Systems (NeurIPS, 2025)*, 2025.

[3] Edward Y. Chang and Longling Geng. Sagallm: Context management, validation, and transaction guarantees for multi-agent llm planning. *Proceedings of the VLDB Endowment (PVLDB)*, 18(12):4874–4886, 2025. https://www.vldb.org/pvldb/vol18/p4874-chang.pdf.

[4] Yinfang Chen, Jiaqi Pan, Jackson Clark, Yiming Su, Noah Zheutlin, Bhavya Bhavya, Rohan Arora, Yu Deng, Saurabh Jha, and Tianyin Xu. STRATUS: A multi-agent system for autonomous reliability engineering of modern clouds. In *Proceedings of the 39th Conference on Neural Information Processing Systems (NeurIPS 2025)*, 2025. https://openreview.net/pdf?id=fYW1PKawwJ.

[5] Yufan Dang, Chen Qian, Xueheng Luo, Jingru Fan, Zihao Xie, Ruijie Shi, Weize Chen, Cheng Yang, Xiaoyin Che, Ye Tian, Xuantang Xiong, Lei Han, Zhiyuan Liu, and Maosong Sun. Multi-agent collaboration via evolving orchestration. In *39th Conference on Neural Information Processing Systems (NeurIPS 2025)*, 2025. https://openreview.net/pdf?id=L0xZPXT3le.

[6] Jiawei Gu, Xuhui Jiang, Zhichao Shi, Hexiang Tan, Xuehao Zhai, Chengjin Xu, Wei Li, Yinghan Shen, Shengjie Ma, Honghao Liu, Saizhuo Wang, Kun Zhang, Yuanzhuo Wang, Wen Gao, Lionel Ni, and Jian Guo. A survey on llm-as-a-judge. *arXiv preprint arXiv:2411.15594*, 2024. https://arxiv.org/abs/2411.15594.

[7] H. T. Kung and John T. Robinson. On optimistic methods for concurrency control. *ACM Transactions on Database Systems (TODS)*, 6(2):213–226, 1981. https://dl.acm.org/doi/10.1145/319566.319567.

[8] LangChain AI. Langgraph: Agent orchestration framework, 2024. https://docs.langchain.com/oss/python/langgraph/persistence.

[9] Thanh Masterman et al. The landscape of emerging ai agent architectures for reasoning, planning, and tool calling: A survey. *arXiv preprint arXiv:2404.11584*, 2024. https://arxiv.org/abs/2404.11584.

[10] Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. GAIA: A benchmark for general ai assistants. In *The Twelfth International Conference on Learning Representations*, 2024. https://openreview.net/pdf?id=fibxvahvs3.

[11] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. Memgpt: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*, 2023. https://arxiv.org/abs/2310.08560.

[12] Sergey Pugachev. Codecrdt: Observation-driven coordination for multi-agent llm code generation. *arXiv preprint arXiv:2510.18893*, 2025. https://arxiv.org/abs/2510.18893.

[13] Viswapriyan Ragupathy. Architectural resilience in ai-driven decision systems under adversarial conditions. In *2026 IEEE 5th International Conference on AI in Cybersecurity (ICAIC)*, pages 1–6, 2026.

[14] Alireza Rezazadeh, Zichao Li, Ange Lou, Yuying Zhao, Wei Wei, and Yujia Bao. Collaborative memory: Multi-user memory sharing in llm agents with dynamic access control. *arXiv preprint arXiv:2505.18279*, 2025. https://arxiv.org/abs/2505.18279.

[15] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. Conflict-free replicated data types. In *13th International Symposium on Stabilization, Safety, and Security of Distributed Systems (SSS '11)*, LNCS 6976, pages 386–400. Springer, 2011. https://dl.acm.org/doi/10.5555/2050613.2050642.

[16] Arunkumar V, G.R. Gangadharan, and Rajkumar Buyya. Agentic Artificial Intelligence (AI): Architectures, taxonomies, and evaluation of large language model agents. *arXiv preprint arXiv:2601.12560*, January 2026. https://arxiv.org/abs/2601.12560.

[17] Shanglin Wu and Kai Shu. Memory in llm-based multi-agent systems: Mechanisms, challenges, and collective intelligence. *TechRxiv preprint*, 2025. https://www.techrxiv.org/doi/full/10.36227/techrxiv.176539617.79044553/v1.

[18] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR 2023)*, 2023. https://openreview.net/pdf?id=WE_vluYUL-X.

[19] Guibin Zhang, Junhao Wang, Junjie Chen, Wangchunshu Zhou, Kun Wang, and Shuicheng Yan. AgenTracer: Who is inducing failure in the LLM agentic systems? In *The Fourteenth International Conference on Learning Representations*, 2026. https://openreview.net/pdf?id=l05DseqvuD.

[20] Jiaru Zou et al. Latent collaboration in multi-agent systems. *arXiv preprint arXiv:2511.20639*, 2025. https://arxiv.org/abs/2511.20639.